

CommitMaster — Microsoft Store Deployment & Admin Guide



Complete Packaging Manual, Microsoft Store Distribution Guide, and Private Admin App Documentation

Project: CommitMaster Desktop

Version: 2.0.0

Author: Saumya Patel

OS Target: Windows 10 / 11 (x64)

1. Overview: The Two Separate Applications

CommitMaster is divided into two distinct applications designed for completely different audiences. Understanding this boundary ensures your personal admin privileges remain secure:

Application	Target Audience	Entry Point / Binary	Purpose & Access Level
Public User App Public Release	End users, developers, team members, public downloaders.	<code>app.py</code> Compiled to: <code>CommitMaster.exe</code>	Standard developer tool: AI commit messages, Git Desktop diff viewer, security inspection, local watched repositories. <i>Zero administrative controls.</i>
Private Admin App Saumya Only	Saumya only (personal operator instance).	<code>admin_app.py</code> Compiled to: <code>CommitMaster-Admin.exe</code>	Full unified operator portal: manage all user accounts, view system activity logs, analyze multi-user commit charts, configure system settings, and inspect database state.

CRITICAL SECURITY RULE: NEVER upload `CommitMaster-Admin.exe` to the Microsoft Store or public GitHub releases. Only upload `CommitMaster.exe` for public distribution. Keep the Admin App exclusively on your private PC.

2. Clickable Single-Click Launchers (No Terminal Needed!)

You do not need to open a terminal or type Python commands to run or build the apps. Three pre-configured single-click desktop helpers are provided in your project root:

Launch_Admin_App.vbs

Double-click to start your Admin App instantly. Runs completely silently in the background with *zero black terminal or command prompt windows flashing.*

Launch_Admin_App.bat

Standard batch launcher fallback. Launches `admin_app.py` using `pythonw.exe`.

Build_All_Packages.bat

Desktop Shortcut Creation

One-click automated compiler. Compiles both `CommitMaster.exe` (User App) and `CommitMaster-Admin.exe` (Admin App) and generates a release ZIP archive in the `dist/` folder.

Right-click `Launch_Admin_App.vbs` → **Show more options** → **Send to** → **Desktop (create shortcut)** to have a permanent 1-click icon on your desktop.

3. What Files Are Compressed Into the Downloadable .EXE

When you double-click `Build_All_Packages.bat`, PyInstaller compresses the Python runtime, graphical engines, and all project modules into self-contained single-file binaries.

Category	What Is Packaged Inside the EXE	Why It Is Included
Python Runtime	<code>python3xx.dll</code> , C-runtime libraries, standard library modules	Allows anyone to run CommitMaster without needing Python installed on their computer.
GUI Engine	Tcl/Tk binaries (<code>tc186t.dll</code> , <code>tk86t.dll</code>) and Tkinter asset library	Powers the custom modern dark-mode desktop window, charts, and smooth scrolling.
Application Modules	The entire <code>commitmaster/</code> module package: • <code>user_dashboard.py</code> & <code>admin_portal.py</code> • <code>navigation.py</code> (Smooth scroll & middle-click autoscroll) • <code>commit_engine.py</code> (Git porcelain & repo integration) • <code>file_inspector.py</code> (Pre-commit security & syntax scanner) • <code>charts.py</code> (Interactive multi-series analytics) • <code>ai_messages.py</code> (AI commit message generation) • <code>database.py</code> (SQLite models & queries)	All business logic, security scanners, user interfaces, and engines are bundled directly inside the executable.
Third-Party Packages	<code>Pillow</code> (PIL), <code>pystray</code> , <code>psutil</code> , <code>requests</code> , <code>urllib3</code>	Background process monitoring, tray icon integration, and network requests.
Application Icon	<code>icon.ico</code>	Windows shell icon, Alt+Tab switcher icon, and taskbar branding.

Runtime Data Files (Stored in AppData):

The local SQLite database (`commitmaster.db`), settings (`config.json`), auto-login tokens (`.session_token`), and log files (`commitmaster.log`) are **not** embedded inside the exe. They are automatically created in `%LOCALAPPDATA%\CommitMaster` on the user's PC, ensuring data persists across updates.

4. Publishing to the Microsoft Store

Microsoft provides two official methods to publish Win32 desktop apps to the Microsoft Store:

Method A: Direct Win32 (.EXE) Submission via URL (Recommended & Easiest)

Since 2021, Microsoft allows developers to submit standard Windows desktop applications directly:

1. Compile the user binary by double-clicking `Build_All_Packages.bat`.
2. Go to your GitHub repository and create a new Release (e.g., `v2.0.0`).
3. Attach `dist/CommitMaster.exe` to the GitHub Release.
4. In **Microsoft Partner Center**, create a new submission and choose **Win32 App**.
5. Enter the download URL of your GitHub release:

```
https://github.com/Saumya-Patel-10/CommitMaster/releases/download/v2.0.0/CommitMaster.exe
```

6. Provide silent installer arguments: `/SILENT` or leave blank for portable exes.
7. Submit for certification. Microsoft's automated bots download, scan, and list the app.

Method B: MSIX Package via MSIX Packaging Tool (Best for Clean Sandbox)

MSIX packages install cleanly like native Windows Store apps and update automatically through the Store:

1. Install the official free **MSIX Packaging Tool** from the Microsoft Store.
2. Open the tool and choose **Application package on this computer**.
3. Select `dist/CommitMaster.exe` as the application executable.
4. Fill in your Package details:
 - **Package Name:** `CommitMaster`
 - **Publisher:** `CN=Your-Partner-Center-Publisher-ID`
 - **Version:** `2.0.0.0`
5. Click **Create Package** → Generates `CommitMaster.msix`.
6. Upload `CommitMaster.msix` directly into Microsoft Partner Center.

5. Testing on the Microsoft Store Before Public Launch

You do not have to release your app to the public immediately. Microsoft Store provides two testing mechanisms:

1. Private Audience (Direct Link Only)

- In Partner Center under **Pricing and availability** → **Visibility**:
- Select **Hidden in the Store. Only customers with a direct link can download**.
- The app is completely invisible in Microsoft Store search results. Only you and users you send the secret link to can install it.

2. Package Flighting (Closed Beta Channel)

- Go to **Package flights** in the left navigation of your app submission.

- Create a flight group named `Beta-Testers` .
- Add your Microsoft account email address (and any friends/testers).
- Upload your test build to this flight. Only members of the group will receive the update through the Windows Store app!
- When testing is complete, click **Promote flight to submission** to roll it out to all users worldwide.

6. Admin App Operator Manual (Your Personal Tool)

Double-clicking `Launch_Admin_App.vbs` opens your unified personal administration dashboard. Key features include:

Section	Capabilities & Controls
User Management	• View all registered user accounts with avatar badges, email addresses, and roles. • Promote users to <code>admin</code> or demote to <code>user</code> . • Activate or deactivate user accounts instantly. • Manually trigger administrative password resets.
Activity Logs & Security	• Complete audit log of all logins, repo scans, pre-commit inspections, and git pushes. • Search and filter logs by username, action type, or date. • Audit security issues caught before commits (secret leaks, merge conflict markers).
Analytics & Charts	• Interactive multi-series commit and scan charts with hover tooltips and clickable legends. • Filter activity by 7 days, 30 days, or custom ranges. • Identify peak committing hours and repository activity.
Global Settings	• Configure default sensitive file patterns (e.g. <code>.env</code> , <code>id_rsa</code> , <code>*.pem</code>). • Set local AI model detection endpoints and fallbacks. • Set background idle thresholds and monitoring intervals.

7. Professional Windows 11 Customization & Visual Assets

CommitMaster has been customized with full Windows 11 native desktop integration:

- **Windows 11 Fluent App Icon:** A high-contrast dark squircle featuring glowing emerald green and cyan Git branch trees, commit nodes, and branding. Compiled directly into `icon.ico` with 16, 24, 32, 48, 64, 128, and 256 mipmap sizes.
- **Windows Taskbar Identity (AppUserModelID):** Registered with `CommitMaster.Desktop.2.0` via Windows shell API, ensuring the taskbar groups CommitMaster under its own branded identity rather than a generic Python launcher.
- **Immersive Dark Titlebar (DWM API):** Native Windows 10/11 titlebars are styled pitch dark (`DWMWA_USE_IMMERSIVE_DARK_MODE`), matching the application palette and eliminating bright white window borders.
- **Microsoft Store Ready Assets:** Pre-generated icons in the `assets/` folder ready for Partner Center: `Square44x44Logo.png` , `Square150x150Logo.png` , and `StoreLogo.png` (300x300).

Summary Checklist:

- ✓ **Run Admin App:** Double-click `Launch_Admin_App.vbs` on your PC (silent background launch).

- ✓ **Build Executables:** Double-click `Build_All_Packages.bat` (compiles both User & Admin binaries).
- ✓ **Deploy to Store:** Submit `dist/CommitMaster.exe` (or `.msix`) to Microsoft Partner Center.